

starfold

A domain-agnostic UMAP+HDBSCAN clustering pipeline with statistical reliability quantification

Andreas W. Neitzel

Instituto de Astrofísica e Ciências do Espaço, Universidade do Porto
ORCID 0000-0001-6283-907X

Documentation, May 2026 (package version 0.0.1)

Abstract

`starfold` packages a four-step unsupervised clustering pipeline as a single primitive, `UnsupervisedPipeline.fit(X)`, returning one immutable result dataclass, `PipelineResult`, that owns the full audit trail of the run. The core methodology follows §3 of Neitzel et al. 2025: `StandardScaler` → UMAP → Optuna-tuned HDBSCAN → trustworthiness, continuity, and a 99.7th-percentile noise baseline. On top of that core, the package ships a small set of paper-silent extensions that compose with the result object: a global 3-sigma credibility test, two modes of input-uncertainty propagation, a hierarchical merge recommender, and chunked-silhouette plus subsample-stability diagnostics. This document is the long-form reference: scope, installation, the public API surface (30 symbols), every methodological choice with citations where the paper is silent, and worked examples with images at every step.

Contents

1	Scope and design	2
1.1	The two load-bearing primitives	2
2	Installation	2
3	Quick start	3
4	The core pipeline	4
4.1	What <code>fit</code> does, step by step	4
4.2	What lives on <code>PipelineResult</code>	4
4.3	Why UMAP beats t-SNE and PCA for this pipeline	4
5	Methodology	4
5.1	UMAP (§3.1)	4
5.2	Trustworthiness and continuity (§3.2)	5
5.3	HDBSCAN and Optuna (§3.3)	5
5.4	Noise baseline (§3.3)	5
5.5	The omnibus 3-sigma credibility test	6
6	Extensions	6
6.1	Input-uncertainty propagation	6
6.2	Hierarchical refinement: <code>suggest_merges</code>	8
6.3	Sub-cluster refit	8
6.4	Robustness diagnostics	8

7 Plotting: two single-call dashboards	9
7.1 Tuning dashboard	9
7.2 Quality dashboard	10
8 Refinement workflow	10
9 Persistence and reproducibility	11
9.1 Save and reload	11
9.2 Determinism	12
10 Public API surface (30 symbols)	12
11 Applied example: stellar populations	13
12 References	13

1 Scope and design

starfold commits to one pipeline shape and exposes it as a single primitive. The package has two concentric scope rings:

Core (paper §3 fidelity). The four-step pipeline implements Neitzel et al. (2025) §3 verbatim. Hyperparameter defaults and statistical thresholds match the paper exactly. This ring is non-negotiable.

Extensions (paper-silent, intentional). On top of the core, starfold ships a small set of validation and refinement primitives that compose with `PipelineResult`: a global credibility test, input-uncertainty propagation, hierarchical refinement, robustness diagnostics, and sample-size-aware defaults. Extensions exist because the paper’s per-cluster threshold and post-hoc workflow leave concrete user questions unanswered.

What the tool is not. It is not a reproduction of the Neitzel et al. (2025) astronomical application. It does not construct stellar samples, apply survey selection functions, or know anything about galactic archaeology. It is not a general-purpose clustering toolkit either: users who want a different pipeline shape should compose `scikit-learn` directly.

1.1 The two load-bearing primitives

- `UnsupervisedPipeline` is the pipeline object. Configure it with UMAP and Optuna kwargs at construction time; call `.fit(X)` once.
- `PipelineResult` is the immutable result dataclass: 2-D embedding, integer cluster labels, per-cluster persistence, trustworthiness, continuity, the noise-baseline summary, the credibility report, the fitted scaler and reducer, the Optuna study, and the run configuration. All post-fit operations are methods on this object or free functions that take it.

2 Installation

starfold is not yet on PyPI. Install from GitHub:

```
# pip
pip install "git+https://github.com/AndreasWNeitzel/starfold.git"

# uv (recommended)
uv add "git+https://github.com/AndreasWNeitzel/starfold.git"
```

Runtime dependencies: `numpy`, `scipy`, `scikit-learn`, `umap-learn`, `hdbscan`, `optuna`, `matplotlib`, `platformdirs`, `joblib`, `tqdm`. Python 3.11 or 3.12. An optional GPU backend via `cuml` is auto-detected when present.

3 Quick start

Any 2-D numpy array goes in; one fit comes out.

```
import numpy as np
import starfold as sf

# X: (n_samples, n_features) numerical matrix. That's all this tool needs.
X = np.random.default_rng(0).normal(size=(3000, 5))

pipeline = sf.UnsupervisedPipeline(
    umap_kwargs={"n_neighbors": 15, "min_dist": 0.0},
    hdbscan_optuna_trials=100,
    random_state=42,
)
result = pipeline.fit(X)

result.embedding # (n_samples, 2) float array
result.labels # (n_samples,) int array, -1 for outliers
result.persistence # (n_clusters,) float array
result.significant # (n_clusters,) bool array vs noise baseline
result.trustworthiness # float in [0, 1]
print(result.summary()) # printable metrics table
result.plot_embedding() # colour-coded by label
result.save("run_01/") # disk snapshot
```

Figure 1 shows the result on `sklearn`'s `digits` dataset (1797 samples, 64 features, 10 true classes). The pipeline recovers the digit classes with no supervision.

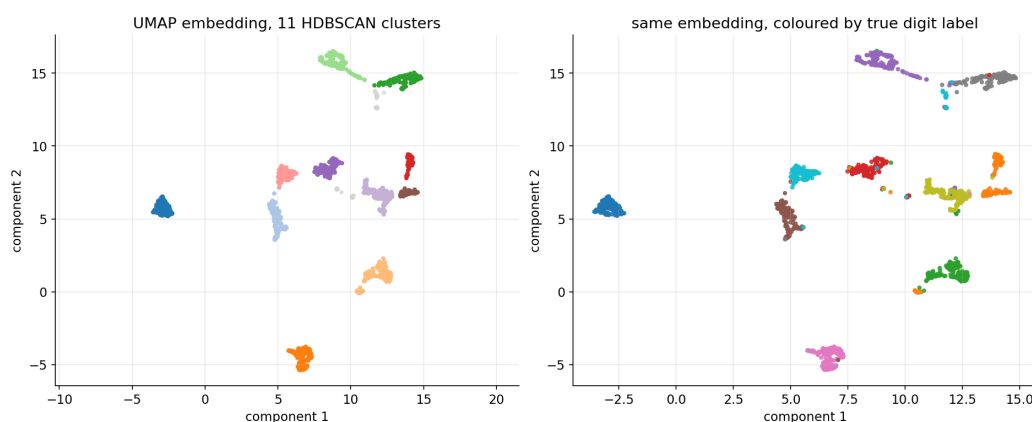


Figure 1: Quickstart output on the `sklearn.datasets.load_digits` dataset: 2-D UMAP embedding coloured by HDBSCAN cluster, outliers in grey. The unsupervised pipeline recovers structure that corresponds closely to the ten true digit classes.

4 The core pipeline

4.1 What fit does, step by step

A single `pipeline.fit(X)` call runs the four-step pipeline in order:

1. **Standardisation.** `StandardScaler` fits and transforms X so every feature has zero mean and unit variance. The scaler is stored on the result for reproducibility.
2. **UMAP embedding.** A 2-D UMAP embedding with `n_neighbors=15`, `min_dist=0.0`, `n_epochs=10 000` by default (paper §3.1). `random_state` propagates.
3. **Optuna-tuned HDBSCAN.** An Optuna TPE search over `min_cluster_size` (log-uniform integer in $[5, 500]$) and `min_samples` (log-uniform in $[1, 50]$) maximises the sum of `cluster_persistence_` across returned clusters. Default budget: 100 trials.
4. **Trustworthiness, continuity, and noise baseline.** Trustworthiness (Venna and Kaski 2001) and its complementary continuity score at $k = n_neighbors$ are computed against `sklearn`'s reference implementation at $atol = 10^{-10}$. The 99.7th-percentile noise baseline is computed from 1000 Gaussian noise realisations; each cluster is flagged **significant** iff its persistence exceeds the threshold.

4.2 What lives on PipelineResult

Beyond the headline arrays listed in Section 3, the result carries the Optuna **study** (for hyperparameter inspection), the fitted UMAP **reducer**, the noise-baseline summary, the optional credibility report, and a **flags** list of diagnostic warnings. Methods on the result:

- `.summary()` → printable metrics table.
- `.silhouette(chunk_size=512)` → chunked silhouette result (per cluster + global, memory-efficient).
- `.suggest_merges()` → cluster-pair merge recommendations from the condensed tree.
- `.refit_subcluster(X, cluster_id)` → second-pass fit on a subset.
- `.propagate_uncertainty(X, sigma, n_draws)` → post-hoc Monte Carlo propagation.
- `.plot_tuning_dashboard()`, `.plot_quality_dashboard(X)` → the two one-call dashboards.
- `.save(path)` → disk snapshot.

4.3 Why UMAP beats t-SNE and PCA for this pipeline

Figure 2 shows `run_umap`, `run_tsne`, and `run_pca` on the same digits data. UMAP delivers the cleanest cluster separation; t-SNE produces visible clusters but with weaker inter-cluster gaps; PCA cannot disentangle the manifold and produces overlapping blobs. The pipeline is built around UMAP, but the alternatives are exposed as standalone helpers for comparison plots.

5 Methodology

This section explains each pipeline step and the paper-silent design choices in detail.

5.1 UMAP (§3.1)

UMAP defaults follow the paper: `n_neighbors=15`, `min_dist=0.0`, `n_epochs=10 000`. The paper notes convergence by ~ 1000 epochs; we keep 10 000 as a default safety margin. The input must be pre-standardised by the caller. `UnsupervisedPipeline` does this internally with `StandardScaler`; `run_umap` on its own does not (it is a thin wrapper).

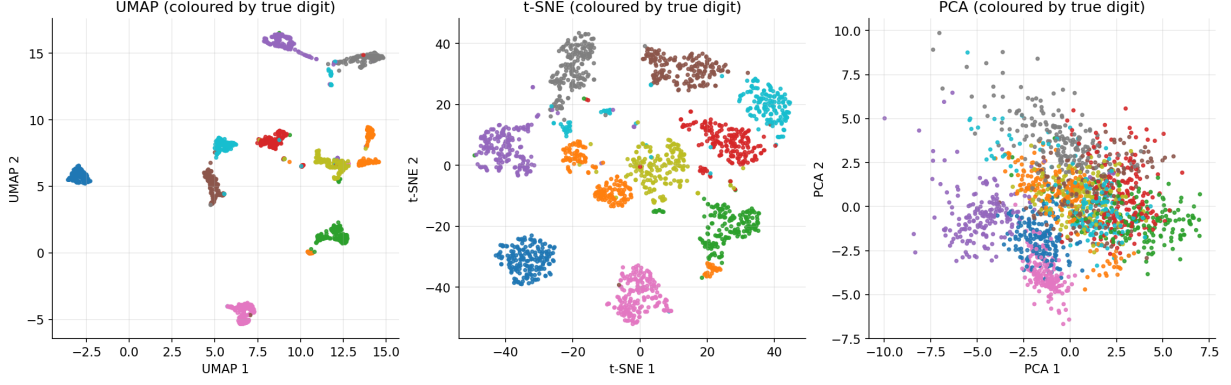


Figure 2: 2-D embeddings of the digits data produced by UMAP, t-SNE, and PCA. UMAP recovers the clearest cluster separation; PCA cannot.

5.2 Trustworthiness and continuity (§3.2)

The trustworthiness score (Venna and Kaski 2001) penalises false neighbours in the embedding:

$$T(k) = 1 - \frac{2}{Nk(2N - 3k - 1)} \sum_{i=1}^N \sum_{x_j \in U_k(x_i)} (r(x_i, x_j) - k), \quad (1)$$

where $U_k(x_i)$ is the set of k nearest neighbours of x_i in the embedding that are not k nearest neighbours of x_i in the input, and $r(x_i, x_j)$ is the rank of x_j ordered by distance to x_i in the input. Continuity is the symmetric measure (false missing neighbours). `starfold`’s implementation is cross-tested against `sklearn.manifold.trustworthiness` at $\text{atol} = 10^{-10}$ across five seeded datasets covering $N \in \{100, 500, 2000, 5000\}$ and $k \in \{5, 15, 30\}$.

The paper’s acceptance heuristic is $T(k = \text{n_neighbors}) > 0.90$. `starfold` reports the value but does not hardcode the threshold.

5.3 HDBSCAN and Optuna (§3.3)

Clustering: `hdbscan.HDBSCAN(min_cluster_size, min_samples, gen_min_span_tree=True)`. The objective is the sum (not the max) of `cluster_persistence_` across returned clusters, so the search rewards both more clusters and more persistent ones.

Search space (paper-silent, our choice). `min_cluster_size` log-uniform integer in $[5, 500]$; on samples where $n/10 < 500$ the upper bound is capped to $\max(5, n/10)$ via `auto_mcs_upper`. `min_samples` log-uniform integer in $[1, 50]$. Sampler: `optuna.samplers.TPESampler(seed=random_state)`. No pruner.

5.4 Noise baseline (§3.3)

The reliability baseline answers “would Gaussian noise of the same shape produce clusters this persistent?”. The procedure:

1. Draw `n_realisations` (default 1000) Gaussian matrices of shape (n, p) .
2. For each, run UMAP with the user’s `umap_kwargs`, then Optuna+HDBSCAN with `per_realisation_trials` (default 20). Record the max cluster persistence.
3. The 99.7th-percentile (3-sigma) of those maxima is the threshold. Each real cluster’s **significant** flag is `persistence > threshold`.

The result is cached on disk under `platformdirs.user_cache_dir("starfold")`, keyed on a hash of $(n, p, \text{frozen_umap_kwargs}, \text{random_state}, n_realisations, \text{per_realisation_trials})$. Figure 3 shows the typical output of the baseline on a clean dataset.

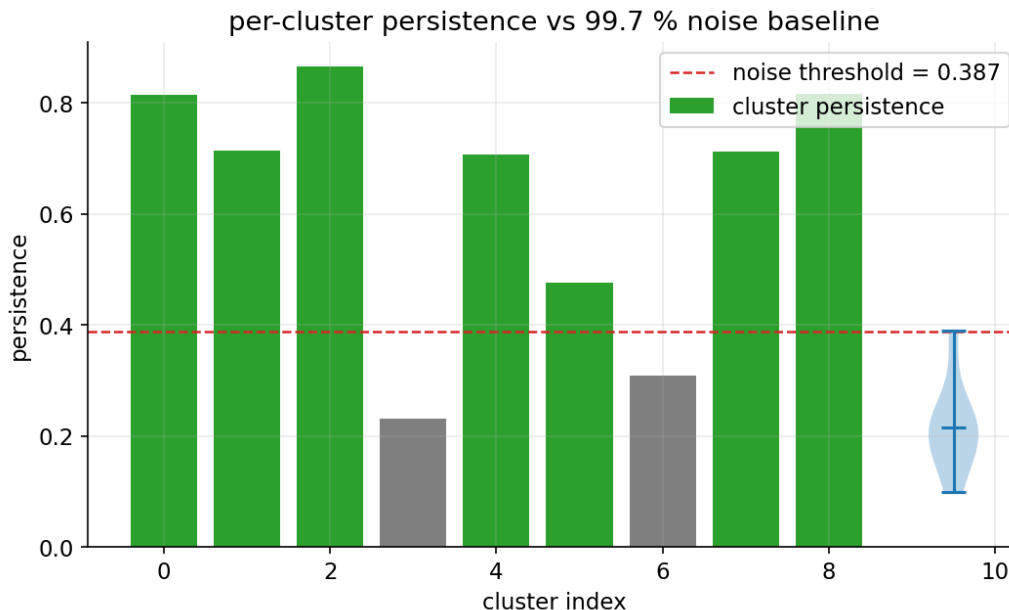


Figure 3: Per-cluster persistence (bars) against the 99.7th-percentile noise baseline (dashed line). Clusters above the line are flagged **significant**; the cluster on the right falls just under threshold.

5.5 The omnibus 3-sigma credibility test

The per-cluster significance flag answers “is this cluster real?”. A separate question is “is the whole run distinguishable from noise?”. `compute_credibility` aggregates three per-run scalars against the noise null:

- `n_clusters` (count of clusters returned).
- `best_objective` (Optuna’s best persistence sum).
- `max_persistence` (single strongest cluster).

Each scalar has a one-sided upper-tail empirical p-value with the Phipson and Smyth (2010) $(r + 1)/(n + 1)$ correction (so a p-value is never exactly zero). The run **passes** at 3-sigma when all three p-values clear $\alpha = 0.003$. The credibility report is `result.credibility`; Figure 4 shows the canonical three-panel visual.

6 Extensions

6.1 Input-uncertainty propagation

When the user’s feature matrix carries per-feature 1-sigma error bars, `starfold` can propagate that uncertainty into a per-sample probability distribution over cluster identities. Two modes:

Mode A: post-hoc Monte Carlo. `result.propagate_uncertainty(X, sigma, n_draws)` freezes the fitted UMAP and HDBSCAN models, draws `n_draws` perturbations of X from $\mathcal{N}(X, \sigma)$ (with `sigma` a scalar, a 1-D vector, or a 2-D (n, p) matrix), and assigns each replica via `umap.transform + hdbscan.approximate_predict`. The output `UncertaintyPropagation` dataclass exposes `membership` of shape $(n, K + 1)$, where `membership[i, k]` is the fraction of

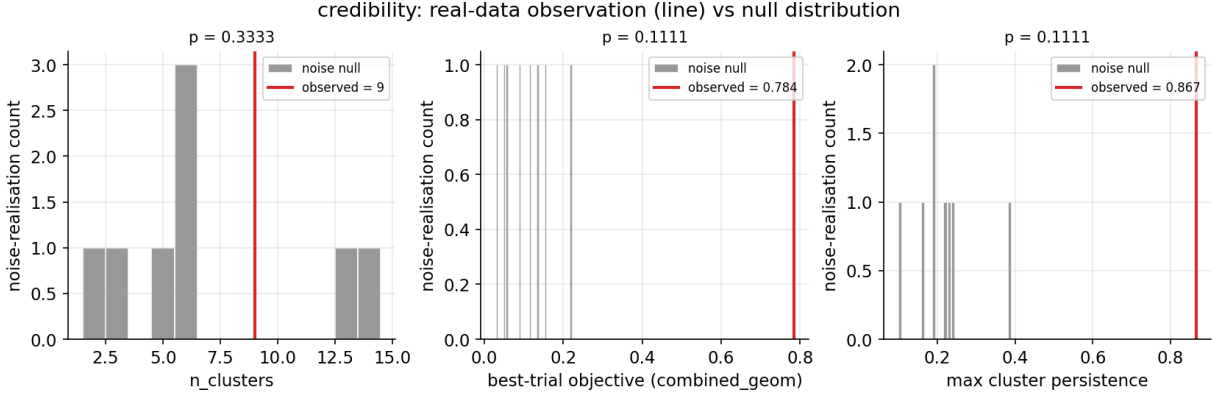


Figure 4: Credibility report: each scalar (top to bottom: cluster count, best objective, max persistence) plotted against its empirical null distribution from the noise realisations. The observed value (vertical line) lands in the upper tail in each case; the omnibus p-value clears 3-sigma.

draws in which sample i landed in cluster k (column K is the outlier fraction), plus `instability` = $1 - \max_k \text{membership}[i, k]$.

Mode B: uncertainty-aware fit. `pipeline.fit_with_uncertainty(X, sigma, n_replicas)` stacks `n_replicas` Gaussian-perturbed copies of X on top of the clean matrix and runs the full pipeline on the augmented $(1 + n_replicas) \cdot n$ row matrix. UMAP and HDBSCAN see the spread itself, so the noise baseline and credibility are computed against the enlarged sample. The returned `UncertaintyAwareFit` bundles the augmented `PipelineResult` with a per-original-sample `UncertaintyPropagation`.

The two modes answer different questions. Mode A audits an existing clustering: “given this fit, how stable is each label?”. Mode B renegotiates the clustering itself: “what structure does the data support when its uncertainty cloud is part of the fitting input?”. The two can disagree directionally and that disagreement is itself a reliability signal.

A topology-stress demonstration. Well-separated blobs are not a good test of uncertainty propagation: as long as σ stays smaller than the inter-cluster gap, the membership matrix is near one-hot. A closed chain of eight Hopf-linked tori (Figure 5) is the natural stress test: adjacent rings thread each other’s holes (linking number ± 1 , verified by `examples.torus_chain.linking_matrix`), so they share a boundary region that uncertainty can blur.

Running `propagate_uncertainty` at three sigma scales ($\times 1$, $\times 5$, $\times 15$ of a baseline per-feature vector) produces a controlled breakdown (Figure 6):

Table 1: Reliability under growing per-feature uncertainty on the eight-torus chain ($N = 2400$ samples, seed 0). The confident-sample fraction degrades monotonically as σ grows. Mean instability is non-monotonic by design: at very large σ the membership mass collapses onto the outlier column (samples become confident outliers), which lowers max-probability ambiguity even as cluster assignment becomes unrecoverable. Read both columns together.

sigma	mean instability	confident (< 0.1)	high-doubt (> 0.5)
$\times 1.0$	0.10	67.2%	1.6%
$\times 5.0$	0.35	10.1%	22.0%
$\times 15.0$	0.27	2.5%	1.3%

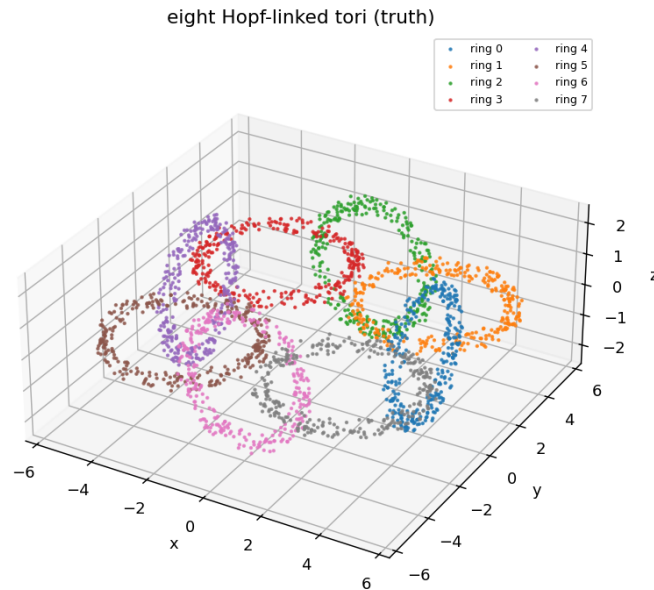


Figure 5: Eight-torus closed Hopf chain in 3D. Alternating vertical and horizontal rings around a backbone circle of radius 4; the linking matrix is tridiagonal ± 1 with corner wraparound (adjacent rings link, non-adjacent rings do not). Augmented with two heteroscedastic “metadata” features to make the pipeline see a 5-D feature matrix.

The confident-sample fraction (instability < 0.1) is the more reliable summary statistic across the sweep: it falls from 67% at $\sigma \times 1$ through 10% at $\times 5$ to 2.5% at $\times 15$. The mean and the high-doubt fraction reverse direction at the largest σ because the failure mode shifts from boundary-blur (mass spread across several clusters) to outlier-collapse (mass concentrated on the outlier column). Both signals carry information; report both alongside the hard label, not just one.

6.2 Hierarchical refinement: `suggest_merges`

After a clean fit, you may suspect HDBSCAN over-split one true cluster into two. `result.suggest_merges()` walks the condensed-tree structure and flags candidate pairs where two clusters are both

- **cohesive** in density (sibling branches in the tree, high `cohesion_ratio`), and
- **geometrically close** in the embedding (`gap_ratio` below threshold).

A merge is **recommended** when both conditions clear default thresholds (`cohesion` ≥ 0.9 , `gap` ≤ 2.0). Figure 7 shows the recommendation panel.

6.3 Sub-cluster refit

The paper’s second-run workflow (refit on a major component) is a scientific choice, not part of the methodology. `result.refit_subcluster(X, cluster_id)` runs a fresh pipeline on the subset of X belonging to `cluster_id`. The output is another `PipelineResult`, with its own embedding, labels, persistence, and noise baseline.

6.4 Robustness diagnostics

Chunked silhouette. The classical silhouette computation builds an $N \times N$ distance matrix and is prohibitive at $N > 10^4$. `chunked_silhouette` computes per-cluster aggregates in chunks of `chunk_size=512` samples by default, keeping peak memory at $\mathcal{O}(N \cdot \text{chunk_size})$. Figure 8 shows the per-cluster silhouette breakdown.

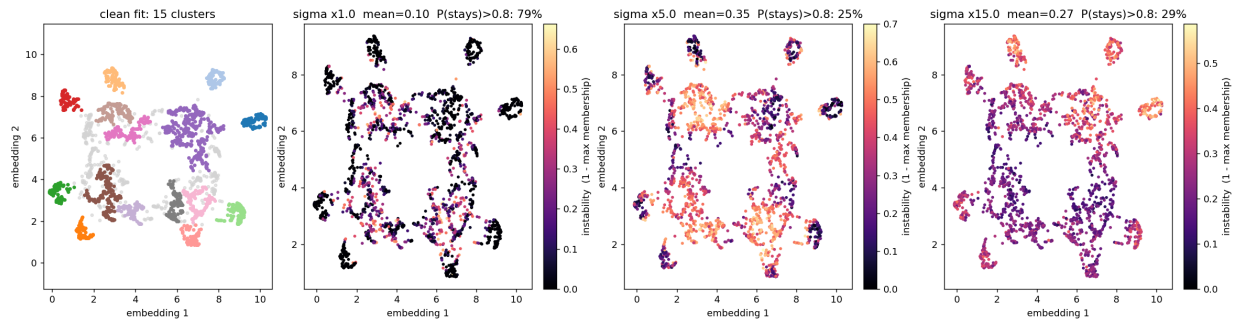


Figure 6: Topology stress test. Left: clean fit embedding coloured by HDBSCAN label (HDBSCAN fragments the chain into more than eight clusters because UMAP cannot embed a torus chain in 2-D without tearing the rings, a known topology artefact, not an uncertainty effect). Right three panels: instability map at σ scaled by 1, 5, 15. Bright regions concentrate at link points between adjacent rings; at the largest σ the outlier column dominates membership and the bright fraction drops accordingly.

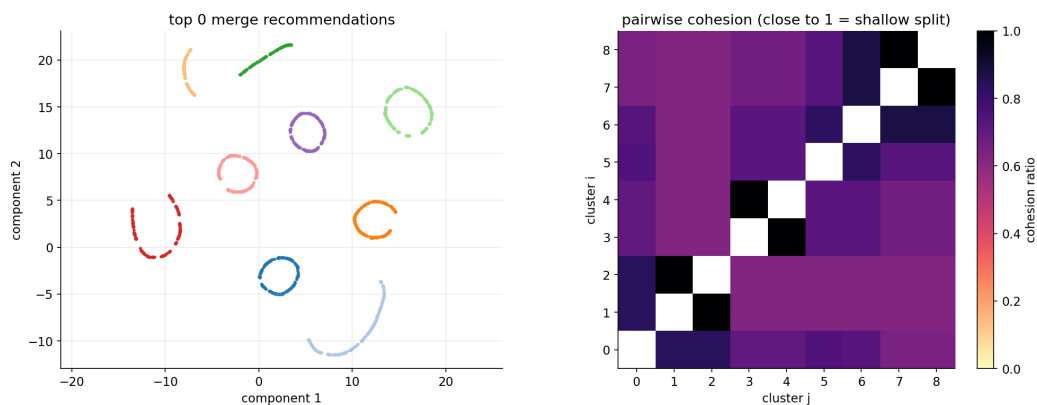


Figure 7: Merge candidate panel. Each candidate pair is plotted in the (cohesion, gap) plane; recommended pairs sit in the upper-left quadrant. The algorithm does not auto-merge; it surfaces the suggestion for the user to act on.

Subsample stability. `compute_subsample_stability` resamples the embedding (default 20 subsamples at 80% fraction), refits HDBSCAN on each, aligns labels via the Hungarian algorithm against the reference fit, and reports the distribution of adjusted Rand index (ARI). A high-mean low-variance ARI distribution is the strongest single signal that the clustering is reproducible under data perturbation.

7 Plotting: two single-call dashboards

`starfold` exposes seven composable plot primitives and bundles the most diagnostic of them into two single-call dashboards on `PipelineResult`.

7.1 Tuning dashboard

`result.plot_tuning_dashboard()` renders eight panels covering the Optuna search:

- optimisation history (best value vs trial),
- parameter importance (fANOVA),
- the 2-D (`min_cluster_size`, `min_samples`) landscape,
- the condensed tree of the final fit,
- the per-cluster persistence bar chart,
- the (objective vs trial) parallel coordinates plot,
- the granularity stability sweep,

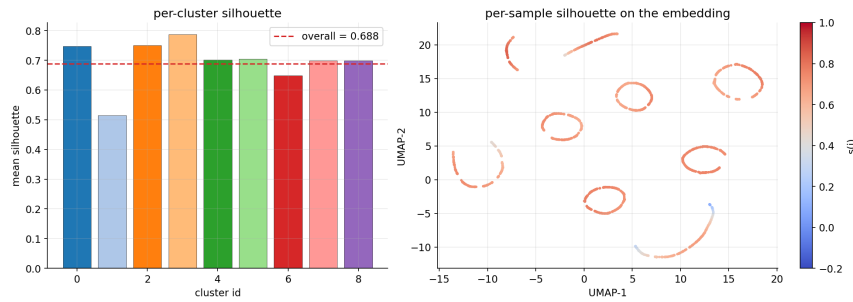


Figure 8: Per-cluster silhouette and overall score. Bars below zero indicate clusters whose internal cohesion is worse than their separation from the nearest neighbour cluster, a candidate for refinement.

- the trustworthiness curve over k .

Figure 9 shows the full assembly.

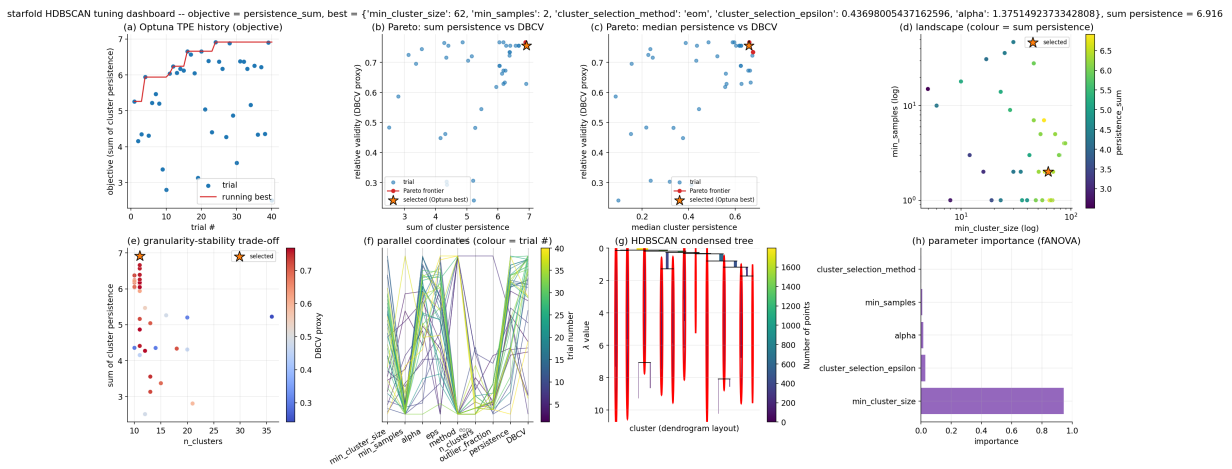


Figure 9: The tuning dashboard. Eight panels on one canvas summarise the Optuna search and the structure of the HDBSCAN fit it selected.

7.2 Quality dashboard

`result.plot_quality_dashboard(X)` renders six panels covering the fitted model:

- membership confidence (when an uncertainty propagation has been attached),
- Optuna fANOVA importance for the final search,
- trustworthiness and continuity curves over k ,
- subsample stability summary (ARI distribution),
- ARI vs subsample, and
- cluster-count vs subsample.

Figure 10 shows the full assembly.

8 Refinement workflow

A typical workflow after a clean fit:

1. Inspect `result.summary()` and the tuning dashboard. Were the chosen hyperparameters at the boundary of the search range?
2. Run `result.silhouette()` and the quality dashboard. Are any clusters below silhouette 0?
3. Inspect `result.suggest_merges()`. Are there cohesive, geometrically-close pairs?

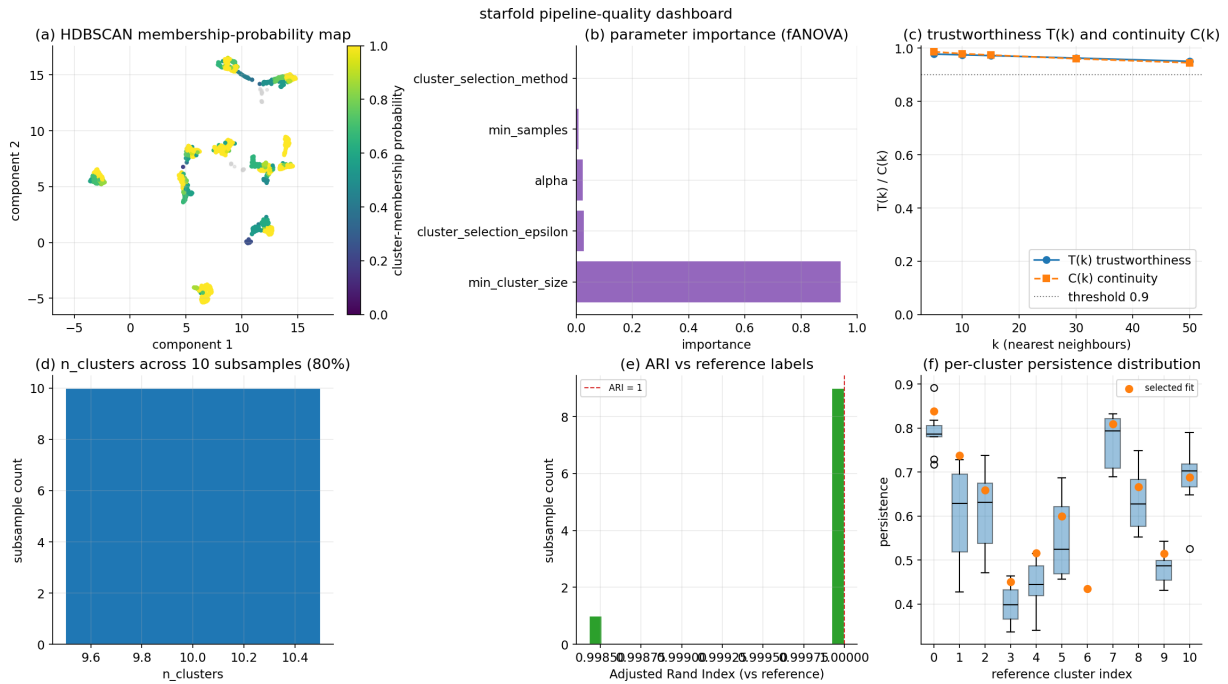


Figure 10: The quality dashboard. Six panels validate the final fit: membership confidence, parameter importance, embedding fidelity, and stability under resampling.

4. If so, decide whether to merge (or to inspect data-domain centroids first).

Figure 11 compares a baseline fit against two refinement strategies on the digits dataset: “blind” (merge the algorithm’s top suggestion) versus “inspection-informed” (look at cluster centroids and merge two clusters that both look like the digit “1”). The inspection-informed strategy improves ARI from 0.870 to 0.891; the blind strategy degrades it. The recommendation flag agrees: the blind pair had `recommended=False`.

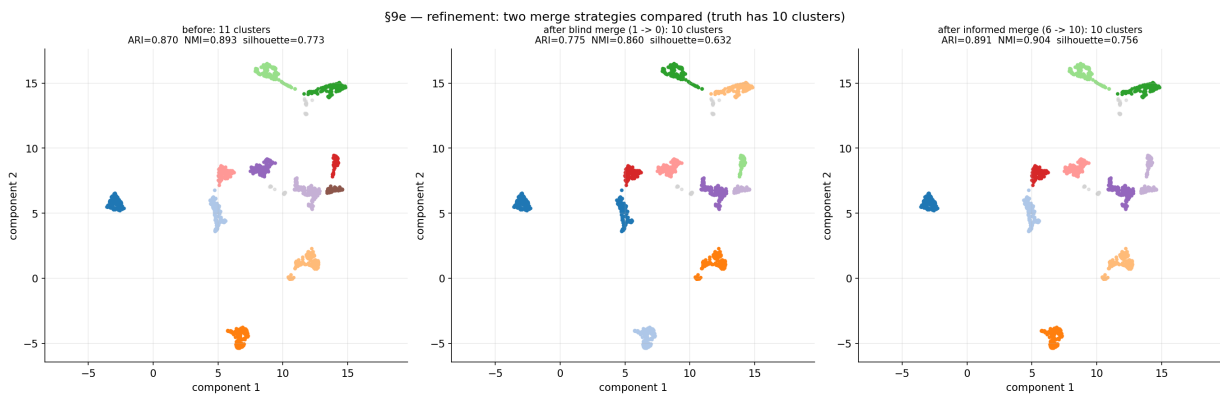


Figure 11: Refinement comparison on `sklearn.datasets.load_digits`. Left: clean fit (ARI 0.870). Centre: blind merge of the top `suggest_merges` candidate (ARI 0.775, degrades). Right: inspection-informed merge of two clusters whose centroid images both look like the digit “1” (ARI 0.891, improves).

9 Persistence and reproducibility

9.1 Save and reload

`result.save(directory)` writes the embedding, labels, persistence, trustworthiness, fitted scaler and UMAP reducer, run configuration, noise-baseline summary, and credibility report into a

directory. `sf.load_pipeline_result(directory)` returns the same content as a plain dict; the Optuna study is not rehydrated (it is stored only as a JSON summary). The numerical-array round-trip is bit-faithful, so a saved fit is the canonical archive of a clustering run.

```
result.save("runs/run_01/")
loaded = sf.load_pipeline_result("runs/run_01/")
assert np.array_equal(loaded["labels"], result.labels)
```

9.2 Determinism

Every stochastic call accepts a `random_state` that propagates to UMAP, the Optuna TPE sampler, the noise-baseline noise generator, and the uncertainty Monte Carlo. With a fixed seed, results are bit-reproducible across runs on the same machine.

10 Public API surface (30 symbols)

starfold's public API, exported from `starfold/__init__.py`:

Symbol	Role
Core entry points	
<code>UnsupervisedPipeline</code>	The pipeline object; one method, <code>.fit(X)</code> .
<code>PipelineResult</code>	Immutable audit trail of a fit.
Core building blocks	
<code>run_umap</code> , <code>run_tsne</code> , <code>run_pca</code>	Embedding wrappers (UMAP is canonical).
<code>trustworthiness</code> , <code>continuity</code>	Single- <i>k</i> scores.
<code>trustworthiness_curve</code> , <code>continuity_curve</code>	Multi- <i>k</i> curves.
<code>run_hdbscan</code>	Single HDBSCAN fit at fixed hyperparameters.
<code>search_hdbscan</code>	Optuna-tuned HDBSCAN.
<code>compute_noise_baseline</code>	99.7th-percentile baseline (cached).
Extension primitives	
<code>compute_credibility</code>	Omnibus 3-sigma test.
<code>propagate_uncertainty</code>	Post-hoc Monte Carlo on a frozen fit.
<code>suggest_merges</code>	Hierarchical merge recommender.
<code>chunked_silhouette</code>	Memory-efficient silhouette.
<code>compute_subsample_stability</code>	Bootstrap-like ARI distribution.
Plot helpers	
<code>plot_embedding</code> ,	Scatter plots and side-by-sides.
<code>plot_embedding_comparison</code>	
<code>plot_trustworthiness_curve</code>	T(<i>k</i>) and C(<i>k</i>) line plot.
<code>plot_condensed_tree</code>	HDBSCAN dendrogram.
<code>plot_optuna_history</code> ,	Optuna diagnostic panels.
<code>plot_optuna_param_importance</code>	
<code>plot_uncertainty_map</code>	Instability scatter on the embedding.
Utilities	
<code>save_pipeline_result</code> ,	Disk round-trip.
<code>load_pipeline_result</code>	
<code>validate_input_matrix</code>	Defensive input checks.
<code>recommend_budget</code> ,	Sample-size-aware defaults.
<code>auto_mcs_upper</code>	
<code>cuml_is_importable</code>	GPU backend probe.

11 Applied example: stellar populations

Tutorial 4 of the bundled notebook arc applies the pipeline to a real chemo-dynamical sample of 9242 Milky Way stars from APOGEE DR19 cross-matched with Gaia DR3 and `galpy`-integrated orbital actions. The package itself has no astronomy dependency; the tutorial uses a pre-built parquet bundled in `docs/data/`. The fit recovers structure that traces the canonical thin disc, thick disc, and halo populations in the $[\text{Fe}/\text{H}]$, $[\alpha/\text{M}]$ chemical plane (Figure 13) and the action space (Figure 14), without ever being told about these reference frames.

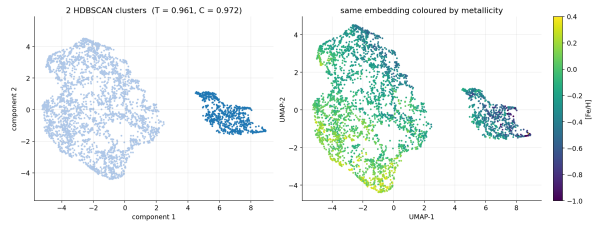


Figure 12: UMAP embedding of 9242 stars, coloured by HDBSCAN cluster.

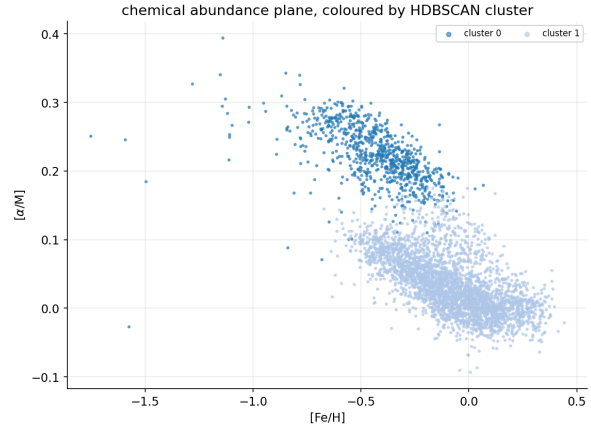


Figure 13: The same clusters projected back onto the chemical plane.

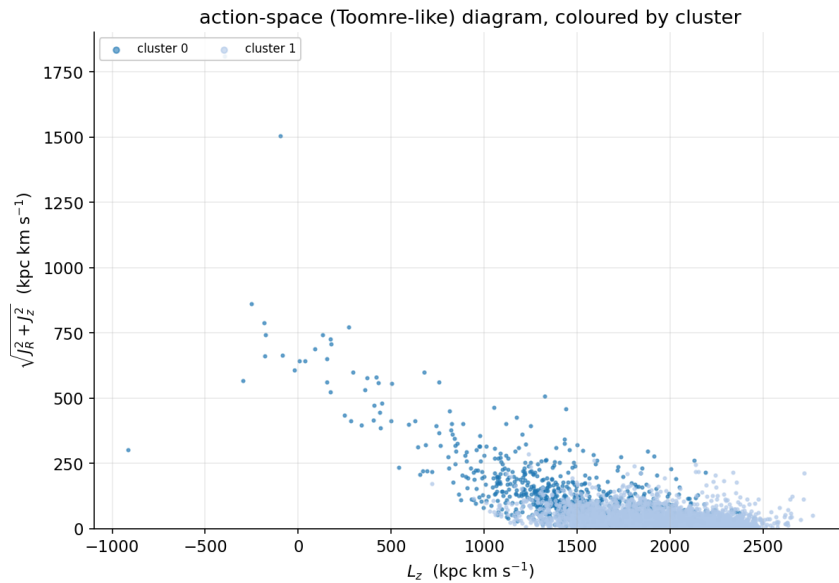


Figure 14: Action-space projection. Disc stars carry circular orbits (low $J_R^2 + J_z^2$, positive L_z); halo stars span the rest of the plane.

12 References

- Neitzel, Campante, Bossini and Miglio (2025). *Astronomy and Astrophysics* **695**, A243. arXiv:2501.16294.
- McInnes, Healy and Melville (2018). UMAP: Uniform Manifold Approximation and Projection. arXiv:1802.03426.

- McInnes, Healy and Astels (2017). hdbscan: Hierarchical density-based clustering. *Journal of Open Source Software* **2**(11), 205.
- Venna and Kaski (2001). Neighbourhood preservation in nonlinear projection methods. ICANN 2001.
- Akiba et al. (2019). Optuna: A next-generation hyperparameter optimization framework. KDD.
- Phipson and Smyth (2010). Permutation p-values should never be zero. *Statistical Applications in Genetics and Molecular Biology* **9**(1).

Citation. If you use `starfold` in published research, please cite the parent paper (Neitzel et al. 2025) and the software via `CITATION.cff`.